Chapter 7 — Arrays

An **array** is JavaScript's ordered list: cart items, search results, database rows, pixels, game moves. Arrays have about forty methods, and knowing which **change** the array and which **return a new one** is the single most useful thing to learn — it prevents a whole class of bugs, especially in React (a UI library) and Redux (a library managing an app's shared data, its *state*).

What you'll learn: why arrays exist and how to create them (literal, `new Array`, `Array.of`, `Array.from`, `fill`); index access, `at()`, `length`, holes, arrays as objects; `push` / `pop` / `unshift` / `shift` and their costs; the full methods table split into **mutating** vs **non-mutating** (incl. ES2023 `toSorted`, `toReversed`, `toSpliced`, `with`); `forEach`, `map`, `filter`, `reduce`, `find` / `findLast`, `some` / `every`, `flat` / `flatMap` and pipelines; sorting with comparators, stability, shuffling; spread/rest/destructuring, shallow vs deep copies, array-likes, recipes; arrays vs objects vs `Map` / `Set`; typed arrays.

Before you start: functions and arrows (Chapter 4) — **callbacks** (functions passed in to be called) are everywhere here; loops (Chapter 3); object spread/destructuring (Chapter 5); `Map`, `Set` and iterators (Chapter 8).

Code is in `code/`; outputs are real Node 24 output.

7.1 Why arrays?

What it is

An **array** stores many values in order under one name. Each value is an **element** with a numeric **index** starting at **0** — like a row of numbered lockers.

The problem it solves

```
const fruit1 = 'apple', fruit2 = 'banana', fruit3 = 'cherry';   // three separate variables...
const fruits = ['apple', 'banana', 'cherry'];                   // ...or one array
```

Separate variables can't be looped, counted, sorted or passed as a group, and a fourth item means new code; an array handles any number with the same code.

How it works

From `code/01_array_basics.mjs`:

```
console.log(fruits, fruits.length, typeof fruits, Array.isArray(fruits), Array.isArray({ length: 0 }));
const mixed = [1, 'two', { three: 3 }, [4], null, () => 5];   // any types, can nest
console.log(mixed.length, mixed[2].three, mixed[3][0], mixed[5]());
```

Output:

```
[ 'apple', 'banana', 'cherry' ] 3 object true false
6 3 4 5
```

- An **array literal** is a comma-separated list in `[ ]`.
- `length` = number of elements (strictly, highest index + 1).
- Arrays are **objects** (`typeof` → `'object'`); test with **`Array.isArray(x)`**.
- Arrays can mix types and nest (arrays of arrays make a grid/**matrix**). Prefer one type per array — clearer and better optimised.

7.2 Creating arrays

Way	Example	Result	When to use
Literal	`['red', 'green', 'blue']`	3 elements	Almost always
`new Array(n)` (one number)	`new Array(3)`	length 3, **no elements** (3 empty slots)	Rarely — prefer `Array.from`
`new Array(a, b, …)`	`new Array('a', 'b', 'c')`	those elements	Never needed — use a literal
`Array.of(...items)`	`Array.of(3)`	`[3]`	When a single number must be an element, not a length
`Array.from(iterableOrArrayLike, mapFn?)` (iterable = anything `for…of` can loop over; array-like = see section 7.7)	`Array.from({ length: 4 }, (_, i) => i * i)`	`[0, 1, 4, 9]`	Build from strings, Sets, Maps, NodeLists (lists of page elements from DOM queries), `arguments`, or generate by index
`Array(n).fill(v)`	`new Array(3).fill(0)`	`[0, 0, 0]`	Pre-filled with a **primitive**
Spread	`[...set]`, `[...'abc']`	copy of any iterable	Copy or convert
`str.split(sep)`	`'a,b'.split(',')`	`['a', 'b']`	From text

```
new Array(3);          // [ <3 empty items> ]  (length 3)
new Array(3, 4);       // [ 3, 4 ]
Array.of(3);           // [ 3 ]
Array.from('hey');     // [ 'h', 'e', 'y' ]
Array(2).fill().map(() => []);   // [ [], [] ]
```

The `new Array` **constructor** (called with `new` to build an object) is inconsistent: **one** number means length, two arguments mean elements — and `new Array(5)` holds five **empty slots** (holes, §7.3), not `undefined`s. `Array.of` / `Array.from` (ES2015) avoid both surprises.

7.3 Reading, changing, adding and removing

Index access, `at()` and `length`

```
const colors = ['red', 'green', 'blue'];
console.log(colors[0], colors[colors.length - 1], colors.at(-1), colors[5]);
colors[2] = 'purple';
console.log(colors);
```

Output:

```
red blue blue undefined
[ 'red', 'green', 'purple' ]
```

Last element: `colors[colors.length - 1]` or **`colors.at(-1)`** (ES2022, negatives from the end). Missing index → `undefined` (no error). Assigning to an index changes the array in place.

The four end-methods

```
const nums = [1, 2, 3];
console.log(nums.push(4), nums.unshift(0), nums);          // both return the NEW length
console.log(nums.pop(), nums.shift(), nums);               // both return the REMOVED element
```

Output:

```
4 5 [ 0, 1, 2, 3, 4 ]
4 0 [ 1, 2, 3 ]
```

Method	Where	Returns	Cost
`push(...items)`	adds at the **end**	new length	O(1) amortised (fast on average; the occasional slow resize is spread over many calls)
`pop()`	removes from the **end**	the removed item (`undefined` if empty)	O(1)
`unshift(...items)`	adds at the **start**	new length	**O(n)** — every element shifts up one index
`shift()`	removes from the **start**	the removed item	**O(n)**

(O(1): time independent of size; O(n): grows with element count — Chapter 17.) `push`/`pop` make a fast **stack** (last in, first out); `shift` as a **queue** (first in, first out) is slow on huge arrays — use an index pointer or a linked structure (each item points to the next, so removing the first moves nothing).

`length` is writable, and holes

```
const letters = ['a', 'b', 'c', 'd'];
letters.length = 2;
console.log(letters);
letters[5] = 'f';                                          // writing past the end creates holes
console.log(letters, letters.length, 3 in letters, letters[3]);

const holey = [1, , 3];
const withUndef = [1, undefined, 3];
let visited = 0; holey.forEach(() => visited++);
console.log(holey, visited, withUndef.map((x) => x ?? 'U'), holey.map((x) => x ?? 'U'));
```

Output:

```
[ 'a', 'b' ]
[ 'a', 'b', <3 empty items>, 'f' ] 6 false undefined
[ 1, <1 empty item>, 3 ] 2 [ 1, 'U', 3 ] [ 1, <1 empty item>, 3 ]
```

- Shrinking `length` **deletes** elements; `arr.length = 0` empties in place (all references see it).
- Writing past the end makes a **sparse array** with **holes** (`3 in letters` is `false`). Holes read as `undefined`; `forEach`, `map`, `filter`, `reduce`, `some`, `every` **skip** them, while `for…of`, `find`, `includes`, `Array.from`, `toSorted`, `with` treat them as `undefined`. Avoid holes — confusing, and engines de-optimise the array (switch to slower general storage).

Looping over an array

```
for (let i = 0; i < colors.length; i++) process.stdout.write(`${i}:${colors[i]} `);  // 0:red 1:green 2:purple
for (const c of colors) process.stdout.write(c + ' ');                               // red green purple
colors.forEach((c, i) => process.stdout.write(`[${i}]${c} `));                       // [0]red [1]green
[2]purple
for (const [i, c] of colors.entries()) process.stdout.write(`${i}=${c} `);           // 0=red 1=green 2=purple
```

Loop	Index?	`break` / `continue` ?	`await` inside works sequentially?	Notes
Classic `for (let i…)`	Yes	Yes	Yes	Most control (step by 2, go backwards)
`for (const x of arr)`	Via `arr.entries()`	Yes	Yes	Cleanest default
`arr.forEach(fn)`	Yes (2nd arg)	**No**	**No** (async callbacks aren't awaited — Chapter 12)	Fine for simple side effects (printing etc.)
`for (const i in arr)`	Yes, as **strings**	Yes	Yes	**Avoid** for arrays: iterates keys, includes inherited enumerable ones (loop-visible properties added to `Array.prototype`)

Arrays are references

```
const a = [1, 2]; const b = a; b.push(3);
console.log(typeof a, a, a === b, [1, 2] === [1, 2]);
```

Output:

```
object [ 1, 2, 3 ] true false
```

As with objects (Chapter 5): assignment copies the reference, `===` compares identity, `const` blocks reassignment but not `push`.

Self-check

1. `[1, 2, 3].push(4)` returns? → `4` (new length).
2. Last element without `length - 1`? → `arr.at(-1)`.
3. `new Array(2)` vs `Array.of(2)`? → two empty slots vs `[2]`.
4. Why avoid `for…in` on arrays? → String keys, visits extra/inherited enumerable properties, not meant for ordered iteration.

7.4 The methods table: mutating vs non-mutating

Why it matters

A **mutating** method changes the array it's called on — fine for a local array, but if it's shared (passed in, in React state, cached) everyone holding a reference sees the change. React compares state by reference, so in-place mutation may mean no **re-render** (redraw). ES2023 added **copying** versions of the classic mutators.

Complete reference

`length` is the only instance **property** you'll use. ✎ = mutates the original.

Method	✎	What it does / returns	Added
`push(...x)` / `pop()`	✎	Add to / remove from end; new length / removed item	ES1
`unshift(...x)` / `shift()`	✎	Add to / remove from start	ES1
`splice(start, deleteCount?, ...items)`	✎	Remove and/or insert anywhere; returns **removed** items	ES1
`sort(compareFn?)`	✎	Sorts in place; returns the same array	ES1
`reverse()`	✎	Reverses in place; returns the same array	ES1
`fill(value, start?, end?)`	✎	Overwrites a range with one value	ES2015
`copyWithin(target, start, end?)`	✎	Copies a range to another position inside the array	ES2015
`toSorted(compareFn?)`		**New** sorted array	**ES2023**
`toReversed()`		**New** reversed array	**ES2023**
`toSpliced(start, deleteCount?, ...items)`		**New** array with the splice applied	**ES2023**
`with(index, value)`		**New** array with one index replaced (negative OK; out of range → `RangeError`)	**ES2023**
`slice(start?, end?)`		Shallow copy of a range (negatives OK)	ES1
`concat(...arraysOrValues)`		New array joining this with others (flattens one level of array args)	ES1
`map(fn)`		New array of `fn` results, same length	ES5
`filter(fn)`		New array of elements where `fn` is truthy	ES5
`flat(depth = 1)`		New array with nested arrays flattened	ES2019
`flatMap(fn)`		`map` then `flat(1)`, in one pass	ES2019
`forEach(fn)`		Calls `fn` on each element; returns `undefined`	ES5
`reduce(fn, init?)` / `reduceRight(fn, init?)`		Folds (combines step by step) to a single value, left→right / right→left	ES5
`find(fn)` / `findIndex(fn)`		First matching element / its index (`undefined` / −1)	ES2015
`findLast(fn)` / `findLastIndex(fn)`		Same, searching from the end	ES2023
`some(fn)` / `every(fn)`		Does any / do all elements pass? (short-circuit)	ES5
`includes(x, from?)`		Boolean; finds `NaN`	ES2016
`indexOf(x, from?)` / `lastIndexOf(x, from?)`		Index using `===` (cannot find `NaN`), or −1	ES5
`at(i)`		Element at `i`; negatives from the end	ES2022
`join(sep = ',')`		String of elements; `null`/`undefined` → empty	ES1
`toString()` / `toLocaleString()`		Comma-joined string / locale-formatted	ES1
`keys()` / `values()` / `entries()`		Iterators (hand out values one at a time, Chapter 8) over indexes / values / `[index, value]` pairs	ES2015
Static: `Array.isArray(x)`, `Array.of(...)`, `Array.from(x, fn?)`, `Array.fromAsync(x)`		Test / create arrays (`fromAsync` awaits async iterables; ES2026, already in Node 24)	ES5–ES2026

(`toSource()` in old lists was a non-standard, now-removed Firefox method — ignore it.)

Mutators vs copying twins in action

From `code/02_mutating_vs_copying.mjs` — `show(label, returnValue, original)` prints the return value, then the original afterwards (the file also demos `reverse`, `fill`, `copyWithin`):

```
let arr = ['a', 'b', 'c', 'd', 'e'];
show('splice(1, 2)', arr.splice(1, 2), arr);                  // remove 2 at index 1
show("splice(1, 0, 'X', 'Y')", arr.splice(1, 0, 'X', 'Y'), arr);  // insert, remove nothing
show("splice(-1, 1, 'Z')", arr.splice(-1, 1, 'Z'), arr);         // replace the last one

arr = [3, 1, 2];
const sorted = arr.sort();
console.log(sorted === arr, arr);

const base = [3, 1, 2];
show('toSorted()', base.toSorted(), base);
show('toReversed()', base.toReversed(), base);
show("toSpliced(1, 1, 'x')", base.toSpliced(1, 1, 'x'), base);
show('with(0, 99)', base.with(0, 99), base);
show('with(-1, 7)', base.with(-1, 7), base);
try { base.with(10, 1); } catch (e) { console.log('with(10, 1) ->', e.constructor.name); }
show('slice(1)', base.slice(1), base);
show('concat([4], 5)', base.concat([4], 5), base);
show('[...base, 4]', [...base, 4], base);
```

Output:

```
splice(1, 2)              ["b","c"]              original: ["a","d","e"]
splice(1, 0, 'X', 'Y')    []                     original: ["a","X","Y","d","e"]
splice(-1, 1, 'Z')        ["e"]                  original: ["a","X","Y","d","Z"]
true [ 1, 2, 3 ]
toSorted()                [1,2,3]                original: [3,1,2]
toReversed()              [2,1,3]                original: [3,1,2]
toSpliced(1, 1, 'x')      [3,"x",2]              original: [3,1,2]
with(0, 99)               [99,1,2]               original: [3,1,2]
with(-1, 7)               [3,1,7]                original: [3,1,2]
with(10, 1) -> RangeError
slice(1)                  [1,2]                  original: [3,1,2]
concat([4], 5)            [3,1,2,4,5]            original: [3,1,2]
[...base, 4]              [3,1,2,4]              original: [3,1,2]
```

`splice` does everything: *where*, *how many* to delete, *what* to insert. `sort` returns **the same array** (`sorted === arr`). The four ES2023 "change array by copy" methods leave the original alone; supported in all current browsers and Node 20+.

A real bug: a helper that mutates its argument

```
function topTwoBad(scores) { return scores.sort((a, b) => b - a).slice(0, 2); }
function topTwoGood(scores) { return scores.toSorted((a, b) => b - a).slice(0, 2); }
const s1 = [50, 90, 70];
console.log(topTwoBad(s1), 'caller now sees', s1);
const s2 = [50, 90, 70];
console.log(topTwoGood(s2), 'caller still sees', s2);
```

Output:

```
[ 90, 70 ] caller now sees [ 90, 70, 50 ]
[ 90, 70 ] caller still sees [ 50, 90, 70 ]
```

The bad version silently reorders the caller's data. Pre-ES2023 fix: `[...scores].sort(...)` / `scores.slice().sort(...)`.

Immutable update cheat sheet (React/Redux style)

Goal	Mutating	Non-mutating
Add to end	`arr.push(x)`	`[...arr, x]`
Add to start	`arr.unshift(x)`	`[x, ...arr]`
Insert at `i`	`arr.splice(i, 0, x)`	`arr.toSpliced(i, 0, x)`
Remove at `i`	`arr.splice(i, 1)`	`arr.toSpliced(i, 1)` or `arr.filter((_, j) => j !== i)`
Remove by id	find index + `splice`	`arr.filter((t) => t.id !== id)`
Replace at `i`	`arr[i] = x`	`arr.with(i, x)`
Update item by id	`item.done = true`	`arr.map((t) => (t.id === id ? { ...t, done: true } : t))`
Sort / reverse	`arr.sort(fn)` / `arr.reverse()`	`arr.toSorted(fn)` / `arr.toReversed()`

Common mistakes

- `sort` / `reverse` / `splice` on React props (a component's inputs from its parent) or state — use the copying twins.
- Confusing `slice` (copy a range) with `splice` (edit in place).
- `arr = arr.push(x)` — `push` returns the new length.
- Thinking `const` prevents `push`; use `Object.freeze(arr)` (shallow) if needed.

7.5 Iteration methods: map, filter, reduce and friends

What it is

These methods call a **callback** per element with `(element, index, array)`, describing *what* to do (**declarative**) rather than *how to loop* (**imperative**).

How it works

From `code/03_iteration_methods.mjs`, with:

```
const orders = [
  { id: 1, customer: 'Ana', total: 120, status: 'paid' },
  { id: 2, customer: 'Ben', total: 35, status: 'pending' },
  { id: 3, customer: 'Ana', total: 60, status: 'paid' },
  { id: 4, customer: 'Cy', total: 250, status: 'refunded' },
];
```

`forEach`, **`map`**, **`filter`**, **`reduce`**:

```
console.log(orders.map((o) => o.total));
console.log(['1', '2', '3'].map(Number), ['1', '2', '3'].map(parseInt)); // classic trap
const paid = orders.filter((o) => o.status === 'paid');
console.log(paid.map((o) => o.id));
const revenue = paid.reduce((sum, o) => sum + o.total, 0);
const byCustomer = orders.reduce((acc, o) => { acc[o.customer] = (acc[o.customer] ?? 0) + o.total; return acc; },
{});
console.log(revenue, byCustomer);
try { [].reduce((a, b) => a + b); } catch (e) { console.log('empty reduce without initial ->',
e.constructor.name); }
console.log(['a', 'b', 'c'].reduceRight((acc, x) => acc + x, ''));
```

Output:

```
[ 120, 35, 60, 250 ]
[ 1, 2, 3 ] [ 1, NaN, NaN ]
[ 1, 3 ]
180 { Ana: 180, Ben: 35, Cy: 250 }
empty reduce without initial -> TypeError
cba
```

Method	Callback returns	Method returns	Think of it as
`forEach`	ignored	`undefined`	"do something with each"
`map`	the new element	new array, **same length**	"transform each"
`filter`	truthy/falsy	new array, **same or fewer** elements	"keep some"
`reduce`	the new accumulator (the running result carried from one step to the next)	the final accumulator	"combine all into one"

`reduce` summing `[120, 60]` from `0`: `0 + 120 → 120`, `120 + 60 → 180`, done → `180`. **Always pass the initial value** — without it the first element is the start, which throws on empty arrays and gives wrong types when building objects.

The `map(parseInt)` trap: `map` passes `(element, index, array)` and `parseInt` takes `(string, radix)`, so it runs `parseInt('2', 1)`, `parseInt('3', 2)` → `NaN`. Use `map((s) => parseInt(s, 10))` or `map(Number)`.

Chaining into a pipeline:

```
const report = orders
  .filter((o) => o.status !== 'refunded')
  .map((o) => ({ ...o, total: o.total * 1.1 }))
  .toSorted((a, b) => b.total - a.total)
  .map((o) => `${o.id}:${o.total.toFixed(2)}`)
  .join(', ');
console.log(report);
```

Output:

```
1:132.00, 3:66.00, 2:38.50
```

Each step returns a new array (originals untouched) and loops once; irrelevant for UI-sized lists, but for millions of rows a single loop or **lazy** iterator helpers (values computed only when asked, Chapter 8) avoid intermediate arrays.

Searching:

```
console.log(orders.find((o) => o.total > 100).id, orders.findIndex((o) => o.total > 100));
console.log(orders.findLast((o) => o.customer === 'Ana').id, orders.findLastIndex((o) => o.customer === 'Ana'));
console.log(orders.find((o) => o.total > 1000), orders.findIndex((o) => o.total > 1000));
console.log(orders.some((o) => o.status === 'refunded'), orders.every((o) => o.total > 0), [].every(() => false));
const vals = [1, 2, NaN, 2];
console.log(vals.includes(NaN), vals.indexOf(NaN), vals.indexOf(2), vals.lastIndexOf(2), vals.includes('2'));
```

Output:

```
1 0
3 2
undefined -1
true true true
true -1 1 3 false
```

Question	Method	Not found
Which element matches?	`find` / `findLast`	`undefined`
At what index?	`findIndex` / `findLastIndex`	`-1`
Is this exact value present?	`includes` (handles `NaN`)	`false`
Where is this exact value?	`indexOf` / `lastIndexOf` (strict `===`, never finds `NaN`)	`-1`
Does any element pass?	`some`	`false` (also for an empty array)
Do all elements pass?	`every`	`true` for an empty array ("vacuous truth")

`find`, `some`, `every` **stop early**. `includes`/`indexOf` use `===`-style equality — `includes('2')` misses the number 2, and objects match only by reference; use `some((o) => o.id === id)`.

Flattening:

```
const nested = [1, [2, [3, [4]]]];
console.log(nested.flat(), nested.flat(2), nested.flat(Infinity));
const sentences = ['hello world', 'good morning'];
console.log(sentences.map((s) => s.split(' ')), sentences.flatMap((s) => s.split(' ')));
console.log([1, 2, 3, 4].flatMap((n) => (n % 2 ? [] : [n, n * 10])));   // filter + map in one pass
```

Output:

```
[ 1, 2, [ 3, [ 4 ] ] ] [ 1, 2, 3, [ 4 ] ] [ 1, 2, 3, 4 ]
[ [ 'hello', 'world' ], [ 'good', 'morning' ] ] [ 'hello', 'world', 'good', 'morning' ]
[ 2, 20, 4, 40 ]
```

`flat(depth)` removes `depth` nesting levels (default 1; `Infinity` = all). `flatMap` maps to *arrays* and joins one level — `[]` drops an element, several items expand it — filter + map in one pass.

Iterators and joining: `[...['x', 'y'].entries()]` → `[ [ 0, 'x' ], [ 1, 'y' ] ]` (`keys()`/`values()` give indexes/values); `[1, [2, 3], null, undefined, 4].join('-')` → `'1-2,3---4'` — `join` makes `null`/`undefined` empty and nested arrays comma-joined. `forEach` can't stop early (`return` ends only that call) — use `for…of` + `break`, or `some`/`find`.

Trade-offs: methods vs loops

	Array methods	`for` / `for…of` loops
Readability	Name the intent (`filter`, `map`)	Generic; intent must be read from the body
Mutation risk	Encourage new arrays	Easy to mutate by accident
Early exit	Only `find`/`some`/`every`	`break` / `return` anywhere
`await` inside	Callbacks are not awaited sequentially	`for…of` + `await` runs in order
Performance	Slight overhead per call; intermediate arrays	Fastest; one pass

Methods by default; loops for early exit, sequential `await` (Chapter 12), or measured performance needs.

Real-world examples

- React lists: `items.map((item) => <Row key={item.id} {...item} />)` (React book).
- API cleanup: `rows.filter(isValid).map(normalize)`.
- Cart total: `cart.reduce((sum, i) => sum + i.price * i.qty, 0)`.
- Feature flags (per-user feature switches): `user.roles.some((r) => r === 'admin')`.

In the interview

> **30-second answer:** "`map` transforms, `filter` selects, `reduce` folds to one value, `find` returns the first match, `some` / `every` answer yes/no with short-circuiting. They take `(element, index, array)` callbacks and don't mutate. I always give `reduce` an initial value and avoid `forEach` when I need `break` or sequential `await`."

- *`map` via `reduce`:* `arr.reduce((acc, x, i) => { acc.push(fn(x, i, arr)); return acc; }, [])`.
- *`['1','2','3'].map(parseInt)` → `[1, NaN, NaN]`?* The index is passed as the radix.
- *`find` vs `filter`?* `find` returns the first element (or `undefined`) and stops; `filter` scans all and returns an array.

Self-check

1. `[1, 2, 3].map((x) => x * 2)`? → `[2, 4, 6]`.
2. `[5, 12, 8].find((x) => x > 6)`? → `12`.
3. `[].every(Boolean)`? → `true`.
4. Sum with `reduce`? → `arr.reduce((s, x) => s + x, 0)`.
5. `[[1], [2, [3]]].flat()`? → `[1, 2, [3]]`.

7.6 Sorting correctly

The problem it solves

`sort()` with no arguments converts elements to **strings** and orders by UTF-16 code units — fine for plain ASCII words, **wrong for numbers**.

From `code/04_sorting.mjs`:

```
console.log([10, 9, 1, 100, 25].sort());                    // default: compared as STRINGS
console.log([10, 9, 1, 100, 25].sort((a, b) => a - b));     // ascending numbers
console.log([10, 9, 1, 100, 25].sort((a, b) => b - a));     // descending numbers
console.log([3, undefined, 1, , 2].sort());                 // undefined and holes go last
```

Output:

```
[ 1, 10, 100, 25, 9 ]
[ 1, 9, 10, 25, 100 ]
[ 100, 25, 10, 9, 1 ]
[ 1, 2, 3, undefined, <1 empty item> ]
```

How it works — the comparator contract

A **comparator** `(a, b) => number` returns **negative** (`a` first), **positive** (`b` first) or **0** (keep relative order). `a - b` works for numbers and Dates (subtraction gives ms). It must be **consistent** — a boolean (`a > b`) is a bug: `false`→0, `true`→1, never negative, so results depend on the engine:

```
console.log('boolean comparator ->', [3, 1, 2].sort((a, b) => a > b));
```

Output:

```
boolean comparator -> [ 3, 1, 2 ]
```

Strings, multiple keys and stability

```
const people = [
  { name: 'Zoe', dept: 'eng', age: 31 },
  { name: 'adam', dept: 'ops', age: 25 },
  { name: 'Émile', dept: 'eng', age: 25 },
  { name: 'Bob', dept: 'ops', age: 31 },
];
console.log(people.toSorted((a, b) => a.age - b.age).map((p) => p.name));
console.log(people.map((p) => p.name).toSorted());                              // code-unit order
console.log(people.map((p) => p.name).toSorted((a, b) => a.localeCompare(b)));  // human order

const byDeptAgeName = (a, b) =>
  a.dept.localeCompare(b.dept) || b.age - a.age || a.name.localeCompare(b.name);
console.log(people.toSorted(byDeptAgeName).map((p) => `${p.dept}/${p.age}/${p.name}`));

const tasks = [['write', 2], ['test', 1], ['deploy', 2], ['plan', 1]];
console.log(tasks.toSorted((a, b) => a[1] - b[1]).map((t) => t[0]));
```

Output:

```
[ 'adam', 'Émile', 'Zoe', 'Bob' ]
[ 'Bob', 'Zoe', 'adam', 'Émile' ]
[ 'adam', 'Bob', 'Émile', 'Zoe' ]
[ 'eng/31/Zoe', 'eng/25/Émile', 'ops/31/Bob', 'ops/25/adam' ]
[ 'test', 'plan', 'write', 'deploy' ]
```

- Strings: `localeCompare` (or a reused `Intl.Collator().compare`) — default order puts capitals first, accents last (Chapter 6).
- **Multi-key sort**: chain with `||` — a tie (0) falls through to the next key; `b.age - a.age` sorts descending.
- **Stability**: since ES2019 `sort` is **stable** — equal elements keep their order (`test` before `plan`), so you can sort by a secondary key, then the primary.

Computed keys, booleans, dates

```
const words = ['banana', 'Kiwi', 'apple', 'fig'];
const sortedByLength = words
  .map((w) => ({ w, key: w.length }))
  .sort((a, b) => a.key - b.key || a.w.localeCompare(b.w))
  .map((x) => x.w);
console.log(sortedByLength);
console.log([true, false, true].toSorted((a, b) => Number(a) - Number(b)));
const dates = [new Date('2024-03-01'), new Date('2023-12-25'), new Date('2024-01-15')];
console.log(dates.toSorted((a, b) => a - b).map((d) => d.toISOString().slice(0, 10)));
```

Output:

```
[ 'fig', 'Kiwi', 'apple', 'banana' ]
[ false, true, true ]
[ '2023-12-25', '2024-01-15', '2024-03-01' ]
```

For expensive keys (parsing dates, normalizing text) compute each **once**, sort by it, strip it — "decorate–sort–undecorate" (Schwartzian transform) — instead of recomputing in the comparator O(n log n) times (the usual cost of a good sort).

Shuffling

```
function shuffle(arr, rand = Math.random) {
  const a = [...arr];
  for (let i = a.length - 1; i > 0; i--) {
    const j = Math.floor(rand() * (i + 1));
    [a[i], a[j]] = [a[j], a[i]];
  }
  return a;
}
```

Fisher–Yates: from the end, swap each position with a random earlier-or-same one — every ordering equally likely, O(n). `arr.sort(() => Math.random() - 0.5)` is **biased** (inconsistent comparator) and slower. (The demo uses a **seeded** generator — same sequence from the same seed — so its output is reproducible: `[ 1, 2, 5, 6, 3, 4 ]`.)

Complexity

V8 uses TimSort; other engines use merge-sort variants (split, sort parts, merge): **O(n log n)**, stable, fast on partly-sorted data.

In the interview

> **30-second answer:** "Default `sort` compares strings, so numbers need `(a, b) => a - b`. The comparator returns negative, zero or positive; chain keys with `||`; strings use `localeCompare`. `sort` mutates and is stable since ES2019; `toSorted` keeps the original."

- *By `lastName`, then `firstName`?* `(a, b) => a.lastName.localeCompare(b.lastName) || a.firstName.localeCompare(b.firstName)`.
- *Why not `sort(() => Math.random() - 0.5)`?* Biased and O(n log n); use Fisher–Yates.

Self-check

1. `[2, 10, 1].sort()`? → `[1, 10, 2]`.
2. Descending numeric comparator? → `(a, b) => b - a`.
3. "Stable"? → Equal elements keep their original relative order.

7.7 Spread, rest and destructuring with arrays

From `code/05_spread_destructuring.mjs`:

```
const a = [1, 2], b = [3, 4];
console.log([...a, ...b], [0, ...a, 2.5, ...b], Math.max(...b), [...'hi'], [...new Set([1, 1, 2])]);

const [first, second, third = 'default', ...rest] = ['x', 'y'];
console.log(first, second, third, rest);
const [, , keep] = ['skip', 'skip', 'take'];
console.log(keep);
let p = 1, q = 2;
[p, q] = [q, p];
console.log(p, q);
const [[r1c1, r1c2], [r2c1]] = [[1, 2], [3, 4]];
console.log(r1c1, r1c2, r2c1);
const { 0: head, length } = ['h', 'e', 'y'];
console.log(head, length);

const minMax = (arr) => [Math.min(...arr), Math.max(...arr)];
const [lo, hi] = minMax([5, 3, 9]);
console.log(lo, hi);
const sum = (...nums) => nums.reduce((s, n) => s + n, 0);
console.log(sum(1, 2, 3), sum(...[10, 20]));
```

Output:

```
[ 1, 2, 3, 4 ] [ 0, 1, 2, 2.5, 3, 4 ] 4 [ 'h', 'i' ] [ 1, 2 ]
x y default []
take
2 1
1 2 3
h 3
6 30
```

- **Spread** `...arr` expands any **iterable** (array, string, Set, Map, generator) in literals and calls.
- **Array destructuring** assigns by **position**; defaults for `undefined`; commas skip; `...rest` gathers the remainder (always an array).
- `[p, q] = [q, p]` swaps.
- Returning an array to destructure is a light multi-value return — React's `const [state, setState] = useState(0)` (a *hook* giving a component a remembered value plus a setter).
- A **rest parameter** `(...nums)` collects arguments into a real array (replacing `arguments` — Chapter 4).

Copies are shallow; the `fill([])` trap

```
const grid = [[0, 0], [0, 0]];
const shallow = [...grid];
const deep = structuredClone(grid);
shallow[0][0] = 9;
console.log(grid[0][0], deep[0][0]);

const bad = Array(3).fill([]);
bad[0].push('x');
const good = Array.from({ length: 3 }, () => []);
good[0].push('x');
console.log(bad, good);
```

Output:

```
9 0
[ [ 'x' ], [ 'x' ], [ 'x' ] ] [ [ 'x' ], [], [] ]
```

`[...arr]`, `slice()`, `concat()`, `Array.from(arr)` copy only the outer array; `structuredClone` copies deeply. `fill(obj)` puts the **same** object in every slot — `Array.from({ length }, () => newThing)` creates one per slot (the standard 2-D grid builder).

Array-likes

An **array-like** has numeric keys and `length` but no array methods — `arguments`, DOM `NodeList` / `HTMLCollection`, strings. Convert with `Array.from` (or spread if iterable), or borrow a method with `call`:

```
const arrayLike = { 0: 'a', 1: 'b', length: 2 };
console.log(Array.from(arrayLike), Array.prototype.map.call(arrayLike, (s) => s.toUpperCase()));
```

Output:

```
[ 'a', 'b' ] [ 'A', 'B' ]
```

Everyday recipes

```
const range = (start, end, step = 1) => Array.from({ length: Math.ceil((end - start) / step) }, (_, i) => start + i * step);
const unique = (arr) => [...new Set(arr)];
const chunk = (arr, size) => Array.from({ length: Math.ceil(arr.length / size) }, (_, i) => arr.slice(i * size, i * size + size));
const zip = (xs, ys) => xs.map((x, i) => [x, ys[i]]);
const last = (arr) => arr.at(-1);
const arraysEqual = (x, y) => x.length === y.length && x.every((v, i) => Object.is(v, y[i]));
```

Output of the demo calls:

```
[ 0, 1, 2, 3, 4 ] [ 10, 7, 4, 1 ]
[ 3, 1, 2 ]
[ [ 1, 2 ], [ 3, 4 ], [ 5 ] ]
[ [ 'a', 1 ], [ 'b', 2 ] ]
3 undefined
true true
```

The last `true` is `[1, 2] == '1,2'` — loose equality stringifies the array. Never compare arrays with `==` / `===`; compare element by element (`arraysEqual`) or, for simple data, `JSON.stringify` results. Grouping (`Object.groupBy`): Chapter 8.

7.8 Arrays vs objects (vs Map/Set)

When to pick which

Use…	When	Example
Array	Order matters; you access by position; you iterate, sort, filter	A playlist, search results, a queue of jobs
Object	A fixed set of named fields describing one thing	`{ name, email, age }` — a record
Array of objects	A list of records — the most common shape of real data	API response `[{ id, title }, …]`
Object / Map keyed by id	Frequent lookup by a key	`usersById[id]` — O(1) instead of `find` O(n)
Set	Uniqueness and fast "is it in?" checks	Visited URLs, selected ids

From `code/01_array_basics.mjs`:

```
const fruitList = ['Apple', 'Banana', 'Cherry'];                    // ordered, by position
const fruitDetails = {                                              // lookup by name
  apple: { color: 'red', taste: 'sweet' },
  cherry: { color: 'red', taste: 'tart' },
};
console.log(fruitList[1], fruitDetails.cherry.taste);
const fruitRecords = [                                              // most common in real apps
  { name: 'Apple', color: 'red' }, { name: 'Banana', color: 'yellow' },
];
console.log(fruitRecords.find((f) => f.name === 'Banana').color);
```

Output:

```
Banana tart
yellow
```

An ordered list is an array; details looked up by name suit an object keyed by name. Real apps hold an **array of objects** and, when id lookups get frequent, build an index: `Object.fromEntries(list.map((x) =>`

`[x.id, x]))` or `new Map(list.map((x) => [x.id, x]))` (Chapter 8).

Common mistakes

- `arr.includes(x)` / `arr.find(...)` inside a loop over another big array — O(n²); build a `Set` / `Map` first.
- Object with numeric keys as a list — no `length`, subtler ordering, no array methods.
- Array as a dictionary (`arr['name'] = …`) — works (arrays are objects) but `length` and methods ignore it.

7.9 Typed arrays (a short note)

What it is

A **typed array** is a fixed-length array of raw numbers of one machine type (8-bit unsigned integers, 32-bit floats, …) over a contiguous memory block, an `ArrayBuffer`; several typed-array **views** can read the same buffer differently. Used for binary data: files, images, audio, WebGL/WebGPU (GPU graphics APIs), WebSockets (always-open two-way browser–server connections), `fetch` bodies, WebAssembly memory (fast compiled code from C/Rust in the browser), and Node's `Buffer` (a `Uint8Array` subclass).

From `code/06_typed_arrays.mjs`:

```
const buf = new ArrayBuffer(8);                 // 8 raw bytes, all zero
const bytes = new Uint8Array(buf);              // view them as 8 unsigned bytes
const words = new Uint16Array(buf);             // ...or as 4 unsigned 16-bit ints (same memory)
bytes[0] = 0x34; bytes[1] = 0x12;
console.log(bytes, words[0].toString(16), '(little-endian on this machine)');

const u8 = new Uint8Array([255, 256, -1, 3.7]);
const clamped = new Uint8ClampedArray([255, 256, -1, 3.7]);
const i8 = new Int8Array([127, 128]);
console.log(u8, clamped, i8);

const f = new Float64Array([3, 1, 2]);
console.log(f.length, typeof f.push, f.map((x) => x * 2), f.toSorted(), Array.isArray(f));
console.log(Float32Array.of(0.1)[0], new Float16Array([0.1])[0]);

console.log(Array.from(f), [...new TextEncoder().encode('Hi')], new TextDecoder().decode(new Uint8Array([72,
105])));
const view = new DataView(buf);
view.setUint16(2, 0xabcd, false);                                   // big-endian write, e.g. a network protocol
console.log(bytes.slice(2, 4), view.getUint16(2, false).toString(16));
```

Output:

```
Uint8Array(8) [
  52, 18, 0, 0,
   0,  0, 0, 0
] 1234 (little-endian on this machine)
Uint8Array(4) [ 255, 0, 255, 3 ] Uint8ClampedArray(4) [ 255, 255, 0, 4 ] Int8Array(2) [ 127, -128 ]
3 undefined Float64Array(3) [ 6, 2, 4 ] Float64Array(3) [ 1, 2, 3 ] false
0.10000000149011612 0.0999755859375
[ 3, 1, 2 ] [ 72, 105 ] Hi
Uint8Array(2) [ 171, 205 ] abcd
```

Type	Element	Range / precision
`Int8Array` / `Uint8Array` / `Uint8ClampedArray`	1 byte	−128…127 / 0…255 (wraps) / 0…255 (clamps and rounds — used by `<canvas>`, the HTML drawing surface, for pixel data)
`Int16Array` / `Uint16Array`	2 bytes	
`Int32Array` / `Uint32Array`	4 bytes	
`Float16Array` (ES2025) / `Float32Array` / `Float64Array`	2 / 4 / 8 bytes	half / single / double precision
`BigInt64Array` / `BigUint64Array`	8 bytes	64-bit integers as BigInt

vs normal arrays: **fixed length** (no `push` / `pop` / `splice`), values **coerced** to the element type (`256` wraps to `0` in `Uint8Array`, `-1` → `255`), `Array.isArray` is `false`. Byte order (**endianness**) follows the machine; `DataView` chooses it explicitly, as binary formats and network protocols require.

Chapter TL;DR

- Arrays are ordered, zero-indexed lists — and objects (`typeof` → `'object'`; `Array.isArray`). Literals; `Array.from({ length }, fn)` / `Array.of` instead of quirky `new Array`.
- `arr[i]` or `arr.at(-1)`; missing index → `undefined`. `length` is writable; avoid holes.
- `push` / `pop` O(1) at the end; `unshift` / `shift` O(n) at the start; `push` returns the new length.
- **Mutators**: `push pop shift unshift splice sort reverse fill copyWithin`. **Copying twins (ES2023)**: `toSorted toReversed toSpliced with`. Non-mutating: `slice concat map filter flat flatMap`, spread. Never mutate shared state/props.
- `map` transforms, `filter` selects, `reduce` folds (pass the initial value), `find` / `findLast` element, `findIndex` index, `some` / `every` short-circuit, `includes` finds `NaN`, `flat` / `flatMap` flatten. `forEach` can't `break` or await.
- Default `sort` is by string; `(a, b) => a - b` for numbers, `localeCompare` for text, chain keys with `||`. Stable since ES2019. Shuffle with Fisher–Yates.
- Spread copies shallowly; `structuredClone` deep; `Array(n).fill([])` shares one object. Destructure by position; swap with `[a, b] = [b, a]`.
- Arrays for ordered lists, objects for records, Map/Set for lookups and uniqueness. Typed arrays: fixed-length binary numbers over an `ArrayBuffer`.